



PROJECT REVIEW

DATE: 18.12.2024

COLLEGE NAME: BANNARI AMMAN INSTITUTE OF TECHNOLOGY

TITLE: MALWARE REVERSE ENGINEERING AND ANALYSIS FOR CYBER ATTRIBUTION

TEAM NAME: CRYPTIC HUNTERS

TEAM MEMBERS:

1. KEERTHANA S M
2. MAHALAKSHMI K
3. HEMANTHSELVA A K

AIM

The primary aim of this project is to develop a user-centric platform for malware reverse engineering and analysis. The platform will allow users to upload files, links, or patterns to identify malicious activity, analyze the nature of the threat, and provide actionable insights. Additionally, the project seeks to educate users on safe practices for handling files and mitigate risks associated with malicious software.

Key Functionalities

1. User Authentication:

- The platform will feature secure **signup** and **login** functionalities to ensure authorized access.

2. File/Link Submission:

- After signing in, users can upload files, images, documents, patterns, links, IPs, or URLs for analysis.
- The system will determine whether the submitted item is malicious or safe.

3. Malicious File Detection and Categorization:

- If the file is **safe**, users will be notified that it poses no threat.
- If the file is **malicious**, detailed information about the malicious activity will be provided.

4. Static and Dynamic Analysis Options:

- Users will choose between **static** or **dynamic analysis** for malicious files:
 - **Static Analysis:** Involves examining the file structure, embedded payloads, and initial infection vectors without executing it.
 - **Dynamic Analysis:** Tracks the behavior of the file during execution, including file modifications, network activity, and system changes.

5. Malware Debugging and Recovery:

- For malicious files, the system will attempt to **debug and convert** the file into a non-malicious, safe version.

6. Threat Attribution:

- The platform will trace the origin of the malicious file, identifying the sender and associated information.

7. User Safety Warning:

- Users will be alerted to avoid downloading untrusted or suspicious files and will receive tailored advice on recognizing potential threats.

Investigating Malware Structure and Functionality

The project will analyze uploaded malicious files to uncover their:

- **Infection Vectors:** How the malware infiltrates systems (e.g., phishing, malicious links).
- **Persistence Mechanisms:** Techniques ensuring continued operation post-reboot or removal attempts.
- **Command-and-Control Communication:** Methods for receiving instructions or exfiltrating data.
- **Payload Delivery:** Actions such as data theft, system compromise, or enabling unauthorized access.

Understanding Malware Behaviors

The platform will study how malware interacts with systems during execution, identifying behaviors such as:

- **Code Obfuscation:** Techniques to hide its true intent.
- **Anti-Debugging:** Detection and circumvention of sandbox or debugger environments.
- **Rootkit Capabilities:** Concealing its presence to maintain control over the system.

User Contribution to Cybersecurity

This project empowers users by:

- Educating them on safe file-handling practices.
- Providing real-time malware analysis and actionable insights.
- Enhancing their awareness of emerging cyber threats.

Long-Term Impact:

This project not only aims to reverse-engineer and analyze malware but also strives to create a **framework for continuous learning and adaptation**. As cyber threats evolve, the techniques and tools developed through this project will serve as a foundation for ongoing improvements in malware detection and analysis methods. By continuously adapting to new malware strains and threat actor tactics, the project will contribute to **building a more secure and resilient digital ecosystem**.

Integration of Tools with Project Workflow

Step 1: User Signup and Login

- **Technology Used:** Express.js (backend) with a user authentication system.
- **Database:** MongoDB/PostgreSQL for storing user credentials.
- **API:** Integration with backend APIs for user authentication and session management.

Step 2: File Upload

- **Frontend:** HTML, CSS, and JavaScript used to create a file upload interface.
- **API:** Integration with VirusTotal API, Hybrid Analysis, and Cuckoo Sandbox for initial file checks.
- **Backend:** Express.js handling incoming file uploads and sending them to external tools for analysis.

Step 3: Check File for Malicious Activity

- **API:** VirusTotal and Hybrid Analysis APIs for reputation checks.
- **Backend:** Node.js or Python backend for communicating with APIs and processing data.
- **Tools:** YARA rules for matching signatures and identifying malicious behavior.

Step 4: Static or Dynamic Analysis

- **Static Analysis Tools:** IDA Pro, Ghidra, OllyDbg, Radare2.
- **Dynamic Analysis Tools:** Cuckoo Sandbox, Wireshark, Procmon.
- **Backend:** Connect the tools to backend for running analysis and displaying results.

Step 5: Safe Message or Malicious Action

- **Frontend:** Display results using JavaScript and HTML.
- **Backend:** Process the results from analysis tools and update the user interface accordingly.

Step 6: Debugging and Normalizing Malicious Files

- **Tools:** Use IDA Pro, Ghidra, or OllyDbg to debug and restore files to a clean state.
- **Backend:** Handle the interaction between the debugging tools and the backend system.

Step 7: Track File Source

- **Database:** Store metadata of uploaded files, including user information.
- **MISP/STIX:** Share IOCs with threat intelligence platforms for correlating file origins.

Step 8: Alert Users about Risks

- **Splunk:** Used for real-time alerts and logs.
- **Notification System:** Push notifications and email alerts integrated to notify users about suspicious files.

Additional Features:

User Dashboard:

After login, users can see their uploaded files, analysis results, and alerts about suspicious activities.

Reporting and Alert System:

Admins can monitor uploaded files and detect suspicious trends or malware patterns.

Multiple Language Access:

Option for using multiple languages by the users.

OBJECTIVE

The primary objective of this project is to develop a user-centric platform for detecting, analyzing, and reverse-engineering malware, allowing users to safely upload files, links, or patterns and determine if they are malicious. The project aims to achieve the following goals:

- **User Authentication and File Submission:**

The first objective is to create a platform where users can securely sign up and log in to upload files, links, IPs, or other indicators for analysis. This step ensures that only authorized users can access the system, and that the files submitted are analyzed for potential threats.

- **Malicious File Detection and Categorization:**

Upon receiving a file or link, the platform will check if it is malicious. If the file is identified as **safe**, the user will be informed, and the file will be marked as non-malicious. If the file is **malicious**, detailed information about the malicious activity (e.g., infection vectors, persistence mechanisms, payload delivery) will be provided to the user.

- **Static and Dynamic Analysis:**

For identified malicious files, the system will allow users to choose between **static** and **dynamic** analysis methods:

- **Static Analysis:** Examining the file's structure, embedded payloads, and infection vectors without executing it.
- **Dynamic Analysis:** Observing the file's behavior during execution in a controlled environment, including system changes, network traffic, and file modifications.

This dual approach helps uncover both the operational mechanisms and the behavioral patterns of the malware.

- **Malware Debugging and Recovery:**

If the file is found to be malicious, the platform will attempt to debug and convert the file into a non-malicious version, allowing users to retrieve a safe version of the file. This approach helps mitigate the risks associated with the file while providing users with a tool for recovery.

- **Attribution of Malicious Activity:**

The system will track the origin of the malicious file, identifying details about the sender (e.g., IP address, file hashes, domain names) to attribute the attack to specific threat actors. This attribution can help distinguish whether the malware is from cybercriminal groups, nation-state actors, or hacktivist entities.

- **User Safety Education:**

A critical goal is to educate users about the risks associated with downloading untrusted files. The platform will provide warnings about suspicious files and offer guidelines for recognizing and avoiding potential threats, thus enhancing overall cybersecurity awareness.

- **Enhancing Detection and Security Frameworks:**

The insights gained from analyzing malicious files will contribute to improving the platform's malware detection capabilities. By identifying malware's tactics, techniques, and procedures (TTPs), the project aims to strengthen the detection systems, reduce false positives, and help users respond to potential threats more effectively.

METHODOLOGY

The methodology for this project is designed to provide a structured and secure approach to malware detection, analysis, and attribution, following a user-centered workflow. The steps outlined below describe the process in detail:

- **User Authentication and File Upload:** The first step in the process involves the user signing up and logging into the platform. After authentication, users can upload various types of files, including images, documents, links, patterns, or IP addresses. These files will be analyzed to determine whether they are malicious or not. The system will provide feedback on the file's status, detailing any malicious activity detected.
- **Malicious File Detection:** Once a file is uploaded, the system will scan the file for known malicious indicators. If the file is deemed **non-malicious**, the user will be notified that the file is safe and no further analysis will be required. If the file is identified as **malicious**, the user will be prompted to proceed with more detailed analysis.
- **Static and Dynamic Analysis Options:** For malicious files, the user will be given the option to choose between **static** or **dynamic** analysis:
 - **Static Analysis:** The file is analyzed without being executed. This approach focuses on examining the file's structure, embedded payloads, and any potential infection vectors.
 - **Dynamic Analysis:** The file is executed in a controlled environment to observe its behavior, including system modifications, network traffic, and interactions with other software. This method helps uncover the operational mechanisms of the malware.
- **File Debugging and Normalization:** In cases where the file is confirmed to be malicious, the system will attempt to debug the file to convert it into a safe, normal version. This step helps users recover from potential infections while ensuring they are not exposed to further harm. The normalized file will be provided to the user for download.
- **Attribution of Malicious Files:** The system will track key indicators of compromise (IOCs), such as IP addresses, file hashes, and domain names, to help attribute the malicious file to a specific threat actor. This step involves collecting detailed information about the origin of the malicious file, including the user who uploaded it and any potential links to known cybercriminal groups or other malicious entities.
- **User Education and Warnings:** The platform will educate users on the risks of downloading untrusted files. Users will receive warnings if they attempt to download any files from untrusted sources, helping to prevent future infections. The system will also provide guidelines on recognizing

suspicious files and links, ensuring users are well-informed about cybersecurity best practices.

- **Safe Environment for Analysis:** To ensure the safety of both the system and user data, all uploaded files will be analyzed in a secure, virtualized environment. The use of virtual machines or sandboxes will prevent any malware from impacting the host operating system or network, ensuring that the analysis process remains isolated and safe.

Performance Optimization

- The performance optimization methodology for this project focuses on improving the efficiency of malware detection, analysis, and attribution, with a strong emphasis on user interaction, file management, and real-time analysis. The following steps describe how performance optimization is achieved:
- **File Upload and Detection Speed:** To ensure a smooth user experience, the system will be optimized to handle fast file uploads and quick detection of malicious files. Optimization techniques such as multi-threading, efficient file parsing, and caching will be employed to reduce latency and ensure rapid analysis, even for large or complex files.
- **Static Analysis:** Static analysis involves analyzing the malware's code without executing it to identify its structure, functions, and vulnerabilities. This process will be enhanced by using optimized tools like Ghidra, IDA Pro, and OllyDbg. The system will prioritize key indicators such as hardcoded IP addresses, obfuscation techniques, and encryption mechanisms to provide a quick overview of potential threats. The static analysis will be automated to reduce processing time and ensure faster results for the user.
- **Dynamic Analysis:** During dynamic analysis, malware is executed in a controlled environment to observe its behavior in real-time. Optimizing this phase involves leveraging efficient tools like **Wireshark** (for network traffic analysis), **Procmon** (for system activity tracking), and **Cuckoo Sandbox** (for automated malware analysis). To minimize resource consumption, the analysis environment will be finely tuned, ensuring that the system does not get overwhelmed by the complexity of running multiple analyses simultaneously.
- **Automation and Scripting:** To streamline the analysis process and reduce manual intervention, automation tools and scripts will be developed. Custom scripts will automate repetitive tasks, such as parsing logs, identifying suspicious activities, and correlating Indicators of Compromise (IOCs). Additionally, machine learning models will be used to enhance the analysis

process by recognizing patterns that are difficult for human analysts to identify. These models will continuously improve over time as new data is fed into the system.

Testing and Evaluation

- **Behavioral Testing:** Once static and dynamic analysis is performed, behavioral testing will be conducted to monitor the file's actions. The system will be optimized to track interactions with files, registry entries, and network connections in real-time. This optimization ensures that the system can handle numerous malware samples simultaneously without sacrificing performance.
- **Impact Evaluation:** The performance of the system in evaluating the full scope of malware impact, such as data exfiltration, system modifications, and compromised user credentials, will be optimized. The analysis tools will be designed to quickly assess the severity of the malware's impact, providing users with actionable insights promptly.
- **Detection Efficacy:** To assess the effectiveness of detection mechanisms, the system will evaluate the performance of existing security tools, such as antivirus software, firewalls, and Intrusion Detection Systems (IDS). The system will be optimized to analyze the behavior of these tools, ensuring that they can accurately detect malware based on IOCs and observed behaviors.

Deployment

- **Integration into Security Systems:** The findings from the malware analysis will be used to optimize existing security systems. This includes improving intrusion detection systems (IDS), antivirus software, and firewalls with new signatures or detection rules. These updates will be automated to ensure that security systems are always up-to-date with the latest threat intelligence, minimizing the time required for manual intervention.
- **Real-Time Monitoring:** Real-time monitoring tools will be optimized to continuously track activities associated with the identified malware behaviors. These tools will be able to detect malicious activity promptly and alert administrators. The system will be fine-tuned to ensure that real-time monitoring does not affect the performance of other systems, allowing for seamless integration into existing infrastructure.
- **Reporting and Alerting:** Automated reporting and alerting systems will be optimized to provide timely and actionable information. The reports will be concise yet comprehensive, detailing the malware's actions and any

identified IOCs. Alerts will be prioritized based on the severity of the threat, ensuring that security teams can respond quickly to critical incidents.

Continuous Improvement

- **Threat Intelligence Sharing:** The system will be optimized to share findings with threat intelligence platforms and collaboration networks efficiently. This includes optimizing the upload and dissemination of IOCs, analysis reports, and insights. By integrating this process into the system workflow, the project will contribute to global threat detection efforts without compromising system performance.
- **Adaptive Defense Mechanisms:** As new malware strains are identified, the defense mechanisms will be continuously updated and optimized. The system will automatically update signatures and detection rules, providing real-time protection against emerging threats. This ensures that the system adapts dynamically to new malware strains.
- **Feedback Loop:** A feedback loop will be established to refine and optimize the malware analysis process. The system will continuously monitor the performance of the analysis and detection techniques, making adjustments as needed. This iterative approach will enhance the overall effectiveness of the security measures over time, ensuring that the system remains capable of handling new and evolving threats efficiently.

TOOLS:

Malware Analysis Tools:

- **VirusTotal:** Used for checking the reputation of files, URLs, and IPs.
- **Hybrid Analysis:** Provides detailed analysis of files and URLs, including behavior analysis.
- **IDA Pro:** A static analysis tool for disassembling and analyzing binary files.
- **Ghidra:** A free software reverse engineering suite used for static analysis and reverse engineering of files.
- **Wireshark:** Used for network traffic analysis during dynamic analysis.
- **Procmon:** Monitors system activities such as file, registry, and process behavior during dynamic analysis.
- **Cuckoo Sandbox:** An automated malware analysis system that provides detailed reports on file behavior in a controlled environment.
- **YARA:** A tool for creating and using custom rules to identify malware based on signatures.

- **Splunk:** Provides real-time monitoring, logging, and alerting for suspicious activities.
- **MISP:** A threat intelligence platform for sharing and correlating IOCs (Indicators of Compromise).
- **STIX/TAXII:** Formats and protocols for exchanging structured threat intelligence data.
- **PEiD:** A tool used to identify packers and obfuscation techniques in files.
- **OllyDbg:** A debugger used for analyzing and debugging executable files.
- **Radare2:** A framework for reverse engineering and analyzing binaries.

Programming Languages and Frameworks Used

Frontend Development:

- **HTML:** Used for creating the structure of web pages.
- **CSS:** Used for styling the web pages and ensuring a responsive design.
- **JavaScript:** Used for creating interactive elements and handling frontend logic, such as file uploads and dynamic content.

Backend Development:

- **Node.js with Express.js:** Used to develop the backend API for handling file uploads, user authentication, and connecting with malware analysis tools.
- **Python (Flask/Django):** An alternative backend framework that can be used to integrate APIs for malware analysis and handle user authentication.

API Integration:

- For making API requests to external tools like VirusTotal, Hybrid Analysis, and others.

Development Environment

Local Development Setup:

- **Node.js:** For backend development and handling API requests.
- **npm/yarn:** For managing package dependencies.
- **Python:** For using Python-based analysis tools like YARA, or Flask/Django for backend development.
- **Git:** For version control and collaboration.
- **VS Code:** The integrated development environment (IDE) used for both frontend and backend development.

Database

Database Technology:

- **MySQL:** Another option for storing structured data, particularly suited for large-scale applications.

Reporting and Alerting

Reporting Tools:

- **Splunk:** Used for monitoring and generating real-time alerts. It can process logs, monitor malicious activity, and trigger notifications based on predefined thresholds or anomalies.
- **Cuckoo Sandbox:** Generates detailed reports about file behavior during dynamic analysis, including registry changes, file system modifications, and network activity.

Conclusion

The integration of various malware analysis tools into this project provides a comprehensive approach to file analysis. By utilizing both static and dynamic analysis techniques, the system can effectively detect, debug, and provide detailed information about potentially malicious files. Additionally, real-time alerts and notifications ensure that users are kept informed about any security threats, making the platform a powerful tool for malware reverse engineering and analysis.

GITHUB REPOSITORY LINKS

- <https://github.com/KEERTHANA169/CRYPTIC-HUNTERS-MALWARE-REVERSE-ENGINEERING-AND-ANALYSIS-FOR-CYBER-ATTRIBUTION>
- <https://github.com/MAHALASHMIK/CRYPTIC-HUNTERS-MALWARE-REVERSE-ENGINEERING-AND-ANALYSING-FOR-CYBER-ATTRIBUTION>
- <https://github.com/HemanthSelva/CRYPTIC-HUNTERS-MALWARE-REVERSE-ENGINEERING-AND-ANALYSING-FOR-CYBER-ATTRIBUTION>